Source sheet: Ensembl (GenomeXplore)

1. Role in GenomeXplore

Ensembl (EMBL-EBI, Europe) provides genome assemblies, annotations and metadata. We use three access points, each with a different role:

Access point	Role
REST search `https://www.ensembl.org/api/search/genomes/v3`	Find genomes from any of 5 key types; get summary stats and the FTP URL
GraphQL `https://www.ensembl.org/data/graphql`	Detailed metadata per genome (assembly, organism, datasets)
FTP `http://ftp.ebi.ac.uk/pub/ensemblorganisms/...`	Location of the heavy files (FASTA, GTF/GFF, ...)

2. REST search endpoint

Request: `GET /api/search/genomes/v3?query=<text>&page=<n>&per_page=<n>`

- `query`: free text. Accepts common name, scientific name, taxonomy id, assembly accession / genome tag, and genome id (UUID).
- `page`, `per_page`: pagination.

Example

URL: `https://www.ensembl.org/api/search/genomes/v3?query=Homo sapiens&page=1&per_page=1`

Response:


```
{
  "meta": {"total_hits": 563, "page": 1, "per_page": 1},
  "matches": [{
    "genome_id": "59871324-7803-4234-856e-2a2bd96d7b3c",
    "genome_tag": "GCA_000001405.29",
    "species_taxonomy_id": "9606",
    "taxonomy_id": "9606",
    "common_name": "Human",
    "scientific_name": "Homo sapiens",
    "type": null,
    "assembly": {
      "accession_id": "GCA_000001405.29",
      "name": "GRCh38.p14",
      "url": "https://identifiers.org/insdc.gca/GCA_000001405.29"
```

```
    },
    "is_reference": true,
    "coding_genes_count": 20543,
    "contig_n50": 54806562,
    "has_variation": true,
    "has_regulation": true,
    "annotation_provider": "GENCODE",
    "organism_id": "1d336185-affe-4a91-85bb-04ebd73cbb56",
    "annotation_method": "Manual annotation",
    "rank": 1,
    "release": {"name": "2026-07", "type": "integrated"},
    "ftp_url":
"http://ftp.ebi.ac.uk/pub/ensemblorganisms/GCA/000/001/405/29/ensembl/
2026_04/"
  }]
}
```

3. GraphQL endpoint

Root queries: `version`, `genome`, `genomes`, plus gene-level queries (`gene`, `genes`, `transcript`, `transcript_search`, `protein_search`, `product`, `overlap_region`, `region`). Only `genome` and `genomes` are needed to find genomes; the others work inside a genome.

3.1 Looking up genomes

Query	Argument	Returns
`genome`	`by_genome_id: {genome_id: "<uuid>"}`	one genome
`genomes`	`by_keyword: {<key>: "<value>"}`	a list of genomes

Keys of `by_keyword`: `tolid`, `assembly_accession_id`, `assembly_name`, `ensembl_name`, `common_name`, `scientific_name`, `scientific_parlance_name`, `species_taxonomy_id` (String), `release_version` (Float).

Rules:

- Exactly one key per query.
- Exact match only (no partial matching); the assembly name includes the patch level, e.g. `GRCh38.p14`.
- The genome tag cannot be searched in GraphQL.

Example

Query:

```
query {
  genome(by_genome_id: {genome_id: "59871324-7803-4234-856e-
2a2bd96d7b3c"}) {
```

```
    genome_id assembly_accession scientific_name taxon_id genome_tag
is_reference
  }
}
```

Response:


```
{
  "data": {
    "genome": {
      "genome_id": "59871324-7803-4234-856e-2a2bd96d7b3c",
      "assembly_accession": "GCA_000001405.29",
      "scientific_name": "Homo sapiens",
      "taxon_id": 9606,
      "genome_tag": "GRCh38",
      "is_reference": true
    }
  },
  "extensions": {
    "execution_time_in_seconds": 1.14
  }
}
```


3.2 Fields returned (type Genome)

`genome_id`, `assembly_accession`, `scientific_name`, `release_number` (Float), `release_date`, `taxon_id`, `parlance_name`, `genome_tag`, `is_reference`, `assembly { assembly_id name accession_id accessioning_body default tolid organism { id scientific_name scientific_parlance_name tol_id is_reference_organism species { taxon_id scientific_name ncbi_common_name alternative_names } } }`, `dataset { dataset_id name type dataset_type source version release release_date release_type }`.

3.3 Which route for which search

User input	Route
common name	GraphQL `common_name` or REST
scientific name	GraphQL `scientific_name` or REST
taxonomy id	GraphQL `species_taxonomy_id` or REST
assembly accession	GraphQL `assembly_accession_id` or REST
genome id	GraphQL `genome(by_genome_id)` or REST
genome tag	REST only

REST search handles all five key types in one call, so it is the proposed single entry point; GraphQL is then called by `genome_id` for the detailed metadata.

4. FTP (heavy files)

The FTP has no API: it is a plain directory listing served over HTTP/FTP. Example for the human reference:

`http://ftp.ebi.ac.uk/pub/ensemblorganisms/GCA/000/001/405/29/ensembl/2026_04/`

The path is built from the assembly accession (`GCA_000001405.29` becomes `GCA/000/001/405/29`), followed by `ensembl` and the release folder. There is no need to rebuild it: the REST search already returns the full URL in the `ftp_url` field, and that is the value stored in the JSON export.

All the files available in a folder can be detected by listing the directory, but this is slow, so it is not ideal for the collector. We only give the `ftp_url`, and users download the raw files they need from there.